TicketStar

Entrega 3: Comunicación síncrona y asíncrona

Informe descriptivo y técnico · Programación Distribuida y Componentes · Proyecto Final

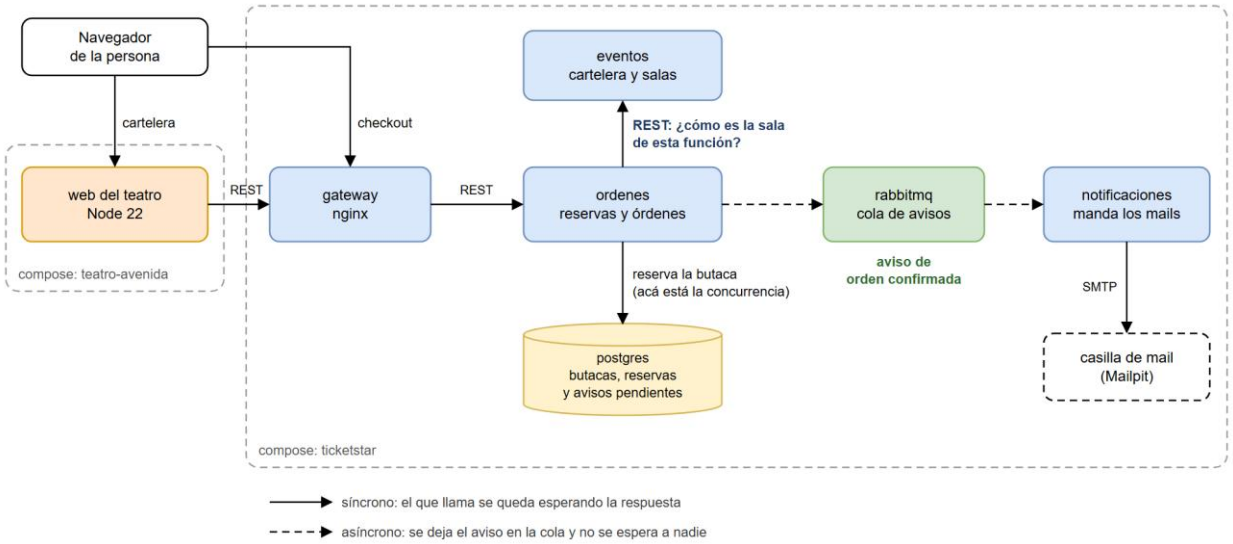
Grupo N°	[completar]
Integrantes	Francisco Aguiar, Rodrigo Rodriguez
Tag / Release	v3-comunicacion
Fecha de entrega	[completar]
Repositorio	https://github.com/franaguiargit/ticketstar

En esta entrega los servicios empezaron a hablarse entre ellos, y con eso ya se puede hacer una compra completa: elegir las butacas, reservarlas, pagar y recibir el mail. Agregamos un canal síncrono (órdenes le pregunta a eventos por REST) y uno asíncrono (órdenes le avisa a notificaciones por una cola de RabbitMQ), y resolvimos el caso de concurrencia que habíamos marcado en la Entrega 1: dos personas que quieren la misma butaca al mismo tiempo. El pago todavía es simulado.

1. Qué hay de nuevo

Pasamos de cinco a ocho contenedores. Esto es lo nuevo y lo que cambió:

Contenedor	Qué hace ahora	
eventos	Además de la cartelera, sabe cómo es cada sala: qué sectores tiene, cuántas filas y butacas, y el precio.	Cambió
ordenes	Arma las butacas de cada función, las reserva, confirma la compra y deja el aviso para notificaciones. El checkout ahora tiene el mapa de butacas.	Cambió
notificaciones	Escucha la cola y manda el mail de cada compra. No tiene API ni puertos.	Nuevo
rabbitmq	La cola de mensajes entre nuestros servicios.	Nuevo
mail	Una casilla de prueba (Mailpit) que hace de servidor de mail de afuera.	Nuevo
web (el teatro)	Suma la página "Mis entradas", que le pide las compras a TicketStar.	Cambió



Cómo se hablan las partes. Las flechas llenas esperan la respuesta y las punteadas son avisos por la cola.

2. El canal síncrono: órdenes le pregunta a eventos

Órdenes es el que maneja las butacas, pero no sabe cómo es cada sala. Eso es de eventos, y cada servicio tiene su propio esquema en la base y no toca las tablas del otro. Entonces, la primera vez que alguien pide el mapa de una función, órdenes llama a eventos por REST (GET /interno/funciones/{id}) y con la respuesta arma una fila por butaca en su propio esquema.

La llamada va directo por la red interna de Docker, sin pasar por el gateway. Las rutas /interno no están publicadas: si alguien las pide desde afuera, el gateway responde 404.

Es síncrona porque órdenes no puede seguir sin esa respuesta. Tiene a una persona esperando el mapa en la pantalla y necesita saber en ese momento si la función existe, si es de ese cliente y qué butacas tiene.

Lo malo de lo síncrono es que si eventos no contesta, órdenes tampoco puede contestar. Para que eso moleste lo menos posible hicimos dos cosas:

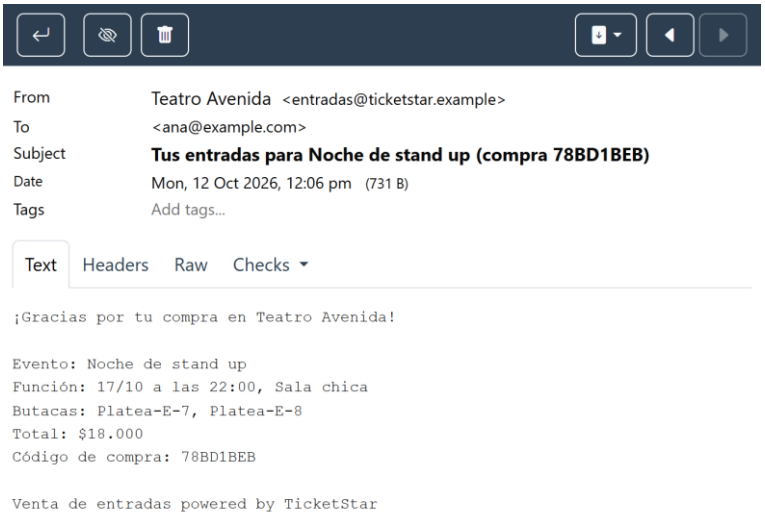
- La llamada tiene un tiempo máximo de 3 segundos. Si eventos no responde, órdenes contesta un 503 con un mensaje claro en lugar de quedarse colgado.
- Órdenes guarda una copia de los datos de la función, así que le pregunta a eventos una sola vez por función. Lo probamos apagando eventos: las funciones que órdenes ya conoce se siguen vendiendo.

La otra comunicación síncrona es la que ya estaba en la Entrega 2: el sitio del teatro y el checkout le hablan a nuestra API por REST, a través del gateway.

3. El canal asíncrono: el aviso de la compra

Cuando una compra se confirma hay que mandarle el mail a la persona. Eso no lo hace órdenes: deja un aviso (orden.confirmada) en RabbitMQ y sigue con lo suyo. El servicio de notificaciones está escuchando esa cola, toma el aviso y manda el mail.

Es asíncrono porque nadie está mirando la pantalla esperando el mail. Si mandarlo tarda o falla, la compra no tiene por qué enterarse. Además así órdenes no necesita saber quién escucha: más adelante facturación va a leer los mismos avisos sin que haya que tocar órdenes.



El mail que manda notificaciones, visto en la casilla de prueba.

Qué pasa si algo está caído

Lo que más nos importaba era que no se pierda ningún aviso y que la compra no dependa de la cola.

- Si notificaciones está apagado, los avisos se juntan en la cola y se mandan cuando vuelve. La cola es durable, o sea que tampoco se pierden si se reinicia RabbitMQ.
- Si RabbitMQ está caído, la compra se confirma igual. El aviso se guarda en una tabla de órdenes en la misma transacción que la orden, y un hilo aparte lo publica cuando RabbitMQ vuelve. Así no puede quedar la orden hecha y el aviso perdido.
- Notificaciones le confirma a RabbitMQ recién después de mandar el mail. Si se cae en el medio, el aviso vuelve a la cola.
- Con todo esto un aviso puede llegar dos veces, así que notificaciones anota los que ya procesó y no manda el mail repetido.

Overview					Messages			Message rates			+/-
Virtual host	**Name**	**Type**	Features	**State**	**Ready**	**Unacked**	**Total**	**incoming**	**deliver / get**	**ack**	
/	**notificaciones**	classic	D Args	running	1	0	1	0.00/s	0.00/s	0.00/s	

La consola de RabbitMQ con notificaciones apagado: hay un aviso esperando en la cola (Ready 1).

Las cuatro cosas las probamos apagando cada servicio en medio de una compra.

4. Por qué cada semántica en cada punto

La regla que usamos es simple: si alguien necesita la respuesta para seguir, es síncrono. Si alcanza con que pase "en un rato", es asíncrono.

Punto del sistema	Cómo	Por qué
El sitio del cliente pide la cartelera o abre una compra	REST, síncrono	El sitio tiene que armar la página con esa respuesta.
La persona reserva butacas en el checkout	REST, síncrono	Necesita un sí o un no en el momento. Si otro llegó antes, hay que avisarle ahí mismo para que elija otra.
Órdenes necesita la sala de una función	REST, síncrono, entre nuestros servicios	Sin ese dato no puede armar el mapa que la persona está esperando.
Se confirmó una compra y hay que mandar el mail	Mensaje en RabbitMQ, asíncrono	Nadie espera el mail mirando la pantalla, y si notificaciones está caído el aviso queda en la cola.
El checkout muestra las butacas que van ocupando los demás	REST cada unos segundos	Por ahora alcanza con volver a pedir el mapa. No hace falta que sea al instante, porque la reserva igual se controla en el servidor.

Todavía no están hechos, pero ya sabemos que van a ser asíncronos: el aviso de la pasarela cuando se aprueba un pago, el webhook con el que le avisamos al cliente que se confirmó una compra y los avisos que va a leer facturación.

5. El caso de concurrencia: dos personas, la misma butaca

Es el que habíamos identificado en la Entrega 1. Cuando se abre la venta, dos personas pueden tocar "Reservar" sobre la misma butaca en el mismo segundo. Y más adelante va a haber varias instancias de órdenes atendiendo a la vez, así que los dos pedidos pueden caer incluso en procesos distintos.

Cómo falla si no se hace nada

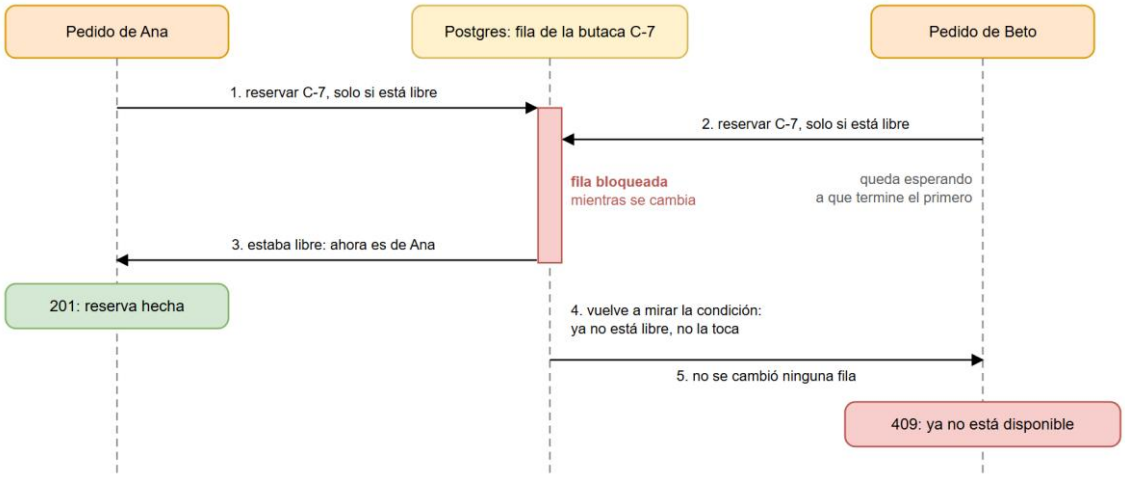
La forma más directa de programar la reserva es en dos pasos: primero consultar si la butaca está libre y después marcarla como reservada. El problema es lo que pasa en el medio. Los dos pedidos consultan, los dos la ven libre y los dos la marcan, y terminan dos personas creyendo que la butaca es suya. Esa versión la dejamos en el código, apagada, para poder mostrar el problema (sección 6).

Cómo lo resolvimos

La reserva es un solo cambio en la base, que se hace solamente si en ese momento la butaca está libre o si la reserva que tenía ya venció:

```
UPDATE butacas SET estado = 'reservada', reserva_id = ..., vence = ...
 WHERE funcion_id = ... AND codigo = ...
   AND (estado = 'libre' OR (estado = 'reservada' AND vence < now()))
```

Postgres bloquea la fila mientras la cambia. Si llegan dos pedidos juntos, el segundo espera a que termine el primero, vuelve a mirar la condición, ve que la butaca ya no está libre y no la toca. Como no cambió ninguna fila, órdenes sabe que perdió y responde 409.



Los dos pedidos sobre la misma butaca, paso a paso.

Elegimos resolverlo en la base y no con un lock en el código de órdenes porque un lock en memoria solo sirve dentro de un proceso. Con dos instancias de órdenes, cada una tendría su propio lock y el problema seguiría estando. La base es el único lugar por el que pasan todas.

Otros casos que salen de lo mismo

- Varias butacas en una reserva: van todas en una transacción. Si una ya no está libre se deshace todo y no queda media reserva hecha. Las butacas se toman siempre en el mismo orden, para que dos reservas que comparten butacas no se queden esperándose una a la otra.
- Doble clic en "Pagar": la confirmación bloquea la reserva antes de tocarla. La segunda confirmación espera, la encuentra ya confirmada y devuelve la misma orden. No se crean dos órdenes ni salen dos mails.
- Reserva vencida: si la persona no paga en 10 minutos, la condición de arriba deja que otro se lleve la butaca. Si el primero quiere pagar después, se le avisa que la reserva venció.

6. Cómo se demuestra

En la carpeta pruebas del repo hay un script, concurrencia.py, que no necesita nada más que Python. En cada ronda elige una butaca libre, abre 20 sesiones de compra y larga los 20 pedidos de reserva al mismo tiempo. Tiene que quedar una sola reserva. La última ronda es con pedidos de dos butacas que se pisan entre sí. Esta es la salida, en una corrida con dos instancias de órdenes atrás del gateway:

```
$ python pruebas/concurrencia.py
Cliente: teatro-avenida · Hamlet, función 1 · 20 pedidos a la vez en cada ronda

Ronda 1: butaca Pullman-G-16  reservas hechas: 1   rechazos (409): 19  OK   (salieron en 4 ms)
Ronda 2: butaca Palcos-B-16   reservas hechas: 1   rechazos (409): 19  OK   (salieron en 8 ms)
Ronda 3: butaca Platea-C-3    reservas hechas: 1   rechazos (409): 19  OK   (salieron en 8 ms)
Ronda 4: butaca Palcos-C-12   reservas hechas: 1   rechazos (409): 19  OK   (salieron en 5 ms)
Ronda 5: butaca Palcos-D-8    reservas hechas: 1   rechazos (409): 19  OK   (salieron en 7 ms)

Ronda de a pares: butacas Pullman-C-1, Platea-H-11 y Palcos-A-2, cada pedido quiere dos
  reservas hechas: 1   rechazos (409): 19   quedaron reservadas: Palcos-A-2, Pullman-C-1   OK

Los pedidos los atendieron 2 instancias de órdenes: ordenes-1 (60), ordenes-2 (60)

RESULTADO: BIEN. En las 6 rondas cada butaca quedó para un solo pedido.
```

Para ver que el problema existe de verdad se puede prender la versión ingenua (con RESERVA_SIN_CONTROL=1, los pasos están en pruebas/README.md) y correr el mismo script. No falla en todas las rondas, porque depende de cómo se crucen los pedidos, pero en varias aparece más de una reserva para la misma butaca:

```
Ronda 1: butaca Pullman-E-18  reservas hechas: 2   rechazos (409): 18  PROBLEMA
         -> 2 pedidos distintos creen que la butaca Pullman-E-18 es suya
Ronda 2: butaca Platea-E-2    reservas hechas: 1   rechazos (409): 19  OK
Ronda 3: butaca Pullman-H-17  reservas hechas: 3   rechazos (409): 17  PROBLEMA
         -> 3 pedidos distintos creen que la butaca Pullman-H-17 es suya
...
RESULTADO: MAL. En 2 de 6 rondas una butaca quedó para más de un pedido.
```

Del lado de la persona hay una prueba con dos navegadores (hecha con Playwright) que eligen la misma butaca y tocan "Reservar" a la vez. Uno pasa a la pantalla de pago y al otro le aparece el aviso, con la butaca ya marcada como ocupada:



Lo que ve la persona que llegó segunda.

7. Qué probamos

Además del script, armamos pruebas con Playwright que usan el sistema desde afuera, como lo usaría un cliente. Lo de apagar servicios lo probamos a mano.

Prueba	Qué revisa
Compra completa en el navegador	Sitio del teatro, checkout, reserva, pago, que llegue el mail y que la compra aparezca en "Mis entradas".
Dos navegadores, la misma butaca	Que una sola persona llegue a la pantalla de pago y la otra vea el aviso.
Cada cliente ve lo suyo	Desde el subdominio del club no se ven ni se pueden tocar las funciones, las reservas ni las órdenes del teatro.
El gateway es la única entrada	Las rutas internas responden 404 desde afuera, y un subdominio que no es de un cliente no entra.
La reserva queda entera o no queda	Si se pide una butaca libre junto con una ocupada, no queda reservada ninguna de las dos.
Apagar servicios en medio de una compra (a mano)	Con notificaciones, RabbitMQ o eventos apagados la compra sigue andando, y los avisos salen cuando el servicio vuelve.

Cómo correr cada una está en pruebas/README.md.

8. Qué cambió respecto de lo planeado

- El pago es simulado: al tocar "Pagar" la compra se confirma. El servicio de pagos con la pasarela viene más adelante, y ahí la confirmación la va a disparar el aviso de la pasarela.
- El mail llega a una casilla de prueba (Mailpit) y no a un servidor de mail real. Todavía no lleva la entrada con el QR.
- Publicamos dos puertos más, pero solo en la propia máquina y para mirar: la consola de RabbitMQ (15672) y la casilla de prueba (8025). No son una entrada al sistema, la API sigue entrando solo por el gateway.
- Por ahora solo se venden funciones con butacas numeradas. El campo general, que va con un contador de lugares, lo hacemos junto con el cliente de prueba del club.
- Si eventos cambia una sala después de que órdenes la copió, órdenes no se entera. La idea es resolverlo con otro aviso por la cola.
- Todavía no hay permisos: cualquiera que llegue al gateway puede pedir las órdenes de un usuario. Eso es parte de la Entrega 5.

9. Qué sigue

- Entrega 4: el contrato de la API en OpenAPI y el descubrimiento de servicios (hoy órdenes encuentra a eventos por un nombre fijo). También arrancamos el segundo cliente de prueba, club-atletico, con la forma B.
- Más adelante: el servicio de pagos, el webhook hacia el cliente y la entrada con el QR.